

COMP2048: Theory of Computation

Dr. Kasra Khosoussi

Week 6

2/4/2026

Recap: T-Decidable vs. T-Recognizable

Turing-Decidable Languages:

- A language L is Turing-*decidable* if there exists a TM M such that for any string $w \in \Sigma^*$:

$$M(w) = \begin{cases} \text{accept} & \text{if } w \in L, \\ \text{reject} & \text{if } w \notin L. \end{cases}$$

- M *halts on all inputs* (i.e., M is called a **decider**)

Recap: T-Decidable vs. T-Recognizable

Turing-Decidable Languages:

- A language L is Turing-*decidable* if there exists a TM M such that for any string $w \in \Sigma^*$:

$$M(w) = \begin{cases} \text{accept} & \text{if } w \in L, \\ \text{reject} & \text{if } w \notin L. \end{cases}$$

- M *halts on all inputs* (i.e., M is called a **decider**)

Turing-Recognizable Languages:

- A language L is Turing-*recognizable* if there exists a TM M such that for any string $w \in \Sigma^*$

$$M(w) = \begin{cases} \text{accept} & \text{if } w \in L, \\ \text{loop forever or reject} & \text{if } w \notin L. \end{cases}$$

- M only needs to halt on strings in L .

Recap: T-Decidable vs. T-Recognizable

Turing-Decidable Languages:

- A language L is Turing-*decidable* if there exists a TM M such that for any string $w \in \Sigma^*$:

$$M(w) = \begin{cases} \text{accept} & \text{if } w \in L, \\ \text{reject} & \text{if } w \notin L. \end{cases}$$

- M **halts on all inputs** (i.e., M is called a **decider**)

Turing-Recognizable Languages:

- A language L is Turing-*recognizable* if there exists a TM M such that for any string $w \in \Sigma^*$

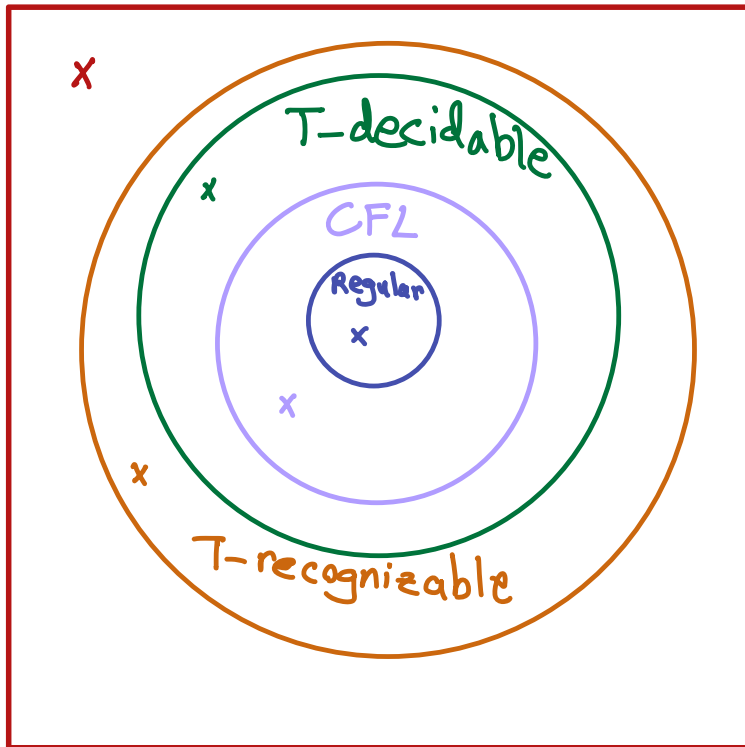
$$M(w) = \begin{cases} \text{accept} & \text{if } w \in L, \\ \text{loop forever or reject} & \text{if } w \notin L. \end{cases}$$

- M only needs to halt on strings in L .

Any decidable language is also recognizable (why?)

Big Picture

x even # of 1's
x $\{a^n b^n \mid n \geq 0\}$
x $\{a^n b^n c^n \mid n \geq 0\}$
x TODAY!
x TODAY!



set of all languages
(OR decision problems)

Limits of Computation

Today's Plan

- ① Counting Argument (proof of existence of unsolvable problems)
- ② Undecidability of Acceptance and Halting Problems (multiple proofs)
- ③ Reducibility
- ④ Big picture

Limits of Computation: Counting Argument

- There are infinitely many TMs and decision problems but . . .

¹Further reading if you're interested: see the formal definition of countable and uncountable sets and Cantor's diagonal argument. Note that Σ^* is countable, but its power set $\mathcal{P}(\Sigma^*)$ (set of all languages/decision problems) is uncountable.

Limits of Computation: Counting Argument

- There are infinitely many TMs and decision problems but . . .
- The set of all Turing machines is **countable** (similar to $\mathbb{N}$, $\mathbb{Z}$, etc)

¹Further reading if you're interested: see the formal definition of countable and uncountable sets and Cantor's diagonal argument. Note that Σ^* is countable, but its power set $\mathcal{P}(\Sigma^*)$ (set of all languages/decision problems) is uncountable.

Limits of Computation: Counting Argument

- There are infinitely many TMs and decision problems but . . .
- The set of all Turing machines is **countable** (similar to $\mathbb{N}$, $\mathbb{Z}$, etc)
- The set of all decision problems is **uncountable** (similar to $\mathbb{R}$)¹

¹Further reading if you're interested: see the formal definition of countable and uncountable sets and Cantor's diagonal argument. Note that Σ^* is countable, but its power set $\mathcal{P}(\Sigma^*)$ (set of all languages/decision problems) is uncountable.

Limits of Computation: Counting Argument

- There are infinitely many TMs and decision problems but . . .
- The set of all Turing machines is **countable** (similar to $\mathbb{N}$, $\mathbb{Z}$, etc)
- The set of all decision problems is **uncountable** (similar to $\mathbb{R}$)¹

¹Further reading if you're interested: see the formal definition of countable and uncountable sets and Cantor's diagonal argument. Note that Σ^* is countable, but its power set $\mathcal{P}(\Sigma^*)$ (set of all languages/decision problems) is uncountable.

Limits of Computation: Counting Argument

- There are infinitely many TMs and decision problems but ...
- The set of all Turing machines is **countable** (similar to $\mathbb{N}$, $\mathbb{Z}$, etc)
- The set of all decision problems is **uncountable** (similar to $\mathbb{R}$)¹

of Turing machines $\ll$ # of decision problems

¹Further reading if you're interested: see the formal definition of countable and uncountable sets and Cantor's diagonal argument. Note that Σ^* is countable, but its power set $\mathcal{P}(\Sigma^*)$ (set of all languages/decision problems) is uncountable.

Limits of Computation: Counting Argument

- There are infinitely many TMs and decision problems but . . .
- The set of all Turing machines is **countable** (similar to $\mathbb{N}$, $\mathbb{Z}$, etc)
- The set of all decision problems is **uncountable** (similar to $\mathbb{R}$)¹

of Turing machines $\ll$ # of decision problems

- We simply don't have enough TMs to solve (recognize/decide) all decision problems (i.e., languages).

¹Further reading if you're interested: see the formal definition of countable and uncountable sets and Cantor's diagonal argument. Note that Σ^* is countable, but its power set $\mathcal{P}(\Sigma^*)$ (set of all languages/decision problems) is uncountable.

Limits of Computation: Counting Argument

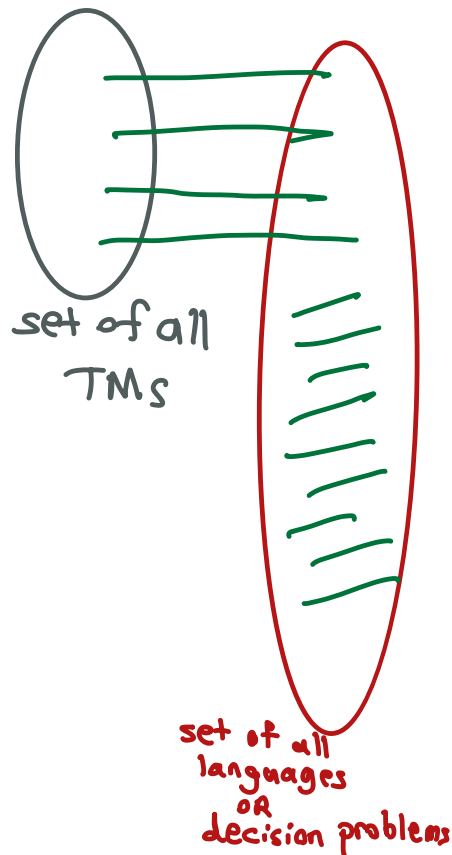
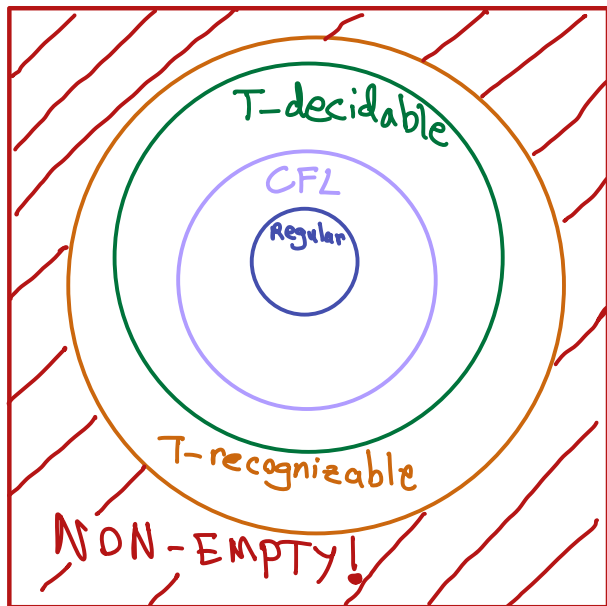
- There are infinitely many TMs and decision problems but ...
- The set of all Turing machines is **countable** (similar to $\mathbb{N}$, $\mathbb{Z}$, etc)
- The set of all decision problems is **uncountable** (similar to $\mathbb{R}$)¹

of Turing machines $\ll$ # of decision problems

- We simply don't have enough TMs to solve (recognize/decide) all decision problems (i.e., languages).
- Therefore, there exist (many!!) decision problems (languages) that cannot be solved (recognized/decided) by *any* TM!

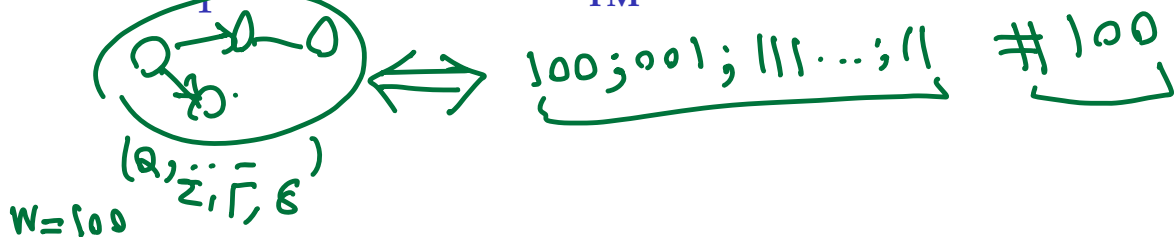
¹Further reading if you're interested: see the formal definition of countable and uncountable sets and Cantor's diagonal argument. Note that Σ^* is countable, but its power set $\mathcal{P}(\Sigma^*)$ (set of all languages/decision problems) is uncountable.

Big Picture



Acceptance and Halting Problems

The Acceptance Problem A_{TM}



$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts input } w \}.$$

- Set of all (encoded) TM-input pairs such that TM **accepts** the input
- Equivalently (yes/no question):

given *any* TM M and *any* input string w , does M accept w ?

- M is a TM
- $\langle M \rangle$ is a string $\in \Sigma^*$
- w is a string $\in \Sigma^*$
- $\langle M, w \rangle = \langle M \rangle \# w \in \Sigma^*$
delimiter ←

The Acceptance Problem is Recognizable

$$A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts input } w\}.$$

Theorem

A_{TM} is recognizable.

- **Proof:**

²Note that when M loops, R will also automatically loop because it's **simulating** M .

The Acceptance Problem is Recognizable

$A_{TM} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts input } w\}.$

Theorem

A_{TM} is **recognizable**.

- **Proof:**

Given $\langle M, w \rangle$, we build another TM R that simulate M on w :

²Note that when M loops, R will also automatically loop because it's **simulating** M .

The Acceptance Problem is Recognizable

$$A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts input } w\}.$$

Theorem

A_{TM} is recognizable.

- **Proof:**

Given $\langle M, w \rangle$, we build another TM R that simulate M on w :

- If M ever accepts, then R accepts.

²Note that when M loops, R will also automatically loop because it's **simulating** M .

The Acceptance Problem is Recognizable

$$A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts input } w\}.$$

Theorem

A_{TM} is recognizable.

• Proof:

Given $\langle M, w \rangle$, we build another TM R that simulate M on w :

- If M ever accepts, then R accepts.
- If M never accepts (by rejecting or looping), then R will loop.²

²Note that when M loops, R will also automatically loop because it's **simulating** M .

The Acceptance Problem is Recognizable

$$A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts input } w\}.$$

Theorem

A_{TM} is recognizable.

• Proof:

Given $\langle M, w \rangle$, we build another TM R that simulate M on w :

- If M ever accepts, then R accepts.
- If M never accepts (by rejecting or looping), then R will loop.²
- By simulating M on w , if $w \in L(M)$ (i.e., if M accepts w), it will halt and accept it in finite time. Therefore, we can detect pairs $\langle M, w \rangle \in A_{\text{TM}}$ and accept them in finite time.

²Note that when M loops, R will also automatically loop because it's **simulating** M .

The Acceptance Problem is Recognizable

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts input } w\}.$$

Theorem

A_{TM} is recognizable.

• Proof:

Given $\langle M, w \rangle$, we build another TM R that simulate M on w :

- If M ever accepts, then R accepts.
- If M never accepts (by rejecting or looping), then R will loop.²
- By simulating M on w , if $w \in L(M)$ (i.e., if M accepts w), it will halt and accept it in finite time. Therefore, we can detect pairs $\langle M, w \rangle \in A_{TM}$ and accept them in finite time.
- We could use the Universal TM to recognize A_{TM}



²Note that when M loops, R will also automatically loop because it's **simulating** M .

The Acceptance Problem is Undecidable (Proof Sketch)

Suppose A_{TM} were decidable ...

```
def H(M_code, w):  
    # returns True if M accepts w, and False otherwise
```



...

```
def D(M_code):  
    if H(M_code, M_code):  
        reject()  
    else:  
        accept()
```



D(D_code)

- If $H(D_code, D_code)$ is True $\rightarrow$ D rejects
- If $H(D_code, D_code)$ is False $\rightarrow$ D accepts

Contradiction. Therefore A_{TM} is undecidable.

The Acceptance Problem is Undecidable

$$A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts input } w\}.$$

Proof: (proof by contradiction)

Suppose A_{TM} was decidable. Then there exists a *decider*³ TM H such that:

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w, \\ \text{reject} & \text{otherwise (if } M \text{ rejects or loops on } w\text{).} \end{cases}$$

³A TM that halts on **all inputs** $\langle M, w \rangle$.

⁴We are asking if M accepts its own encoding $w = \langle M \rangle$.

The Acceptance Problem is Undecidable

$$A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts input } w\}.$$

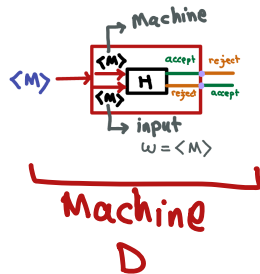
Proof: (proof by contradiction)

Suppose A_{TM} was decidable. Then there exists a *decider*³ TM H such that:

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w, \\ \text{reject} & \text{otherwise (if } M \text{ rejects or loops on } w\text{).} \end{cases}$$

Now let's build **another TM** that takes as input $\langle M \rangle$, runs H on $\langle M, \langle M \rangle \rangle$,⁴ and **negates** the output. Note that this is well-defined because H was a decider (i.e., always halts in a finite time). Therefore, the new TM D :

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } H \text{ rejects } \langle M, \langle M \rangle \rangle, \\ \text{reject} & \text{if } H \text{ accepts } \langle M, \langle M \rangle \rangle. \end{cases}$$



Note that D is clearly a *decider* (because H was a decider).

³A TM that halts on **all inputs** $\langle M, w \rangle$.

⁴We are asking if M accepts its own encoding $w = \langle M \rangle$.

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w, \\ \text{reject} & \text{otherwise (if } M \text{ rejects or loops on } w \text{).} \end{cases}$$

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } H \text{ rejects } \langle M, \langle M \rangle \rangle, \\ \text{reject} & \text{if } H \text{ accepts } \langle M, \langle M \rangle \rangle. \end{cases}$$

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w, \\ \text{reject} & \text{otherwise (if } M \text{ rejects or loops on } w\text{).} \end{cases}$$

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } H \text{ rejects } \langle M, \langle M \rangle \rangle, \\ \text{reject} & \text{if } H \text{ accepts } \langle M, \langle M \rangle \rangle. \end{cases}$$

Now let's see what happens if we run D on $\langle D \rangle$ (i.e., on its encoding!).

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w, \\ \text{reject} & \text{otherwise (if } M \text{ rejects or loops on } w \text{).} \end{cases}$$

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } H \text{ rejects } \langle M, \langle M \rangle \rangle, \\ \text{reject} & \text{if } H \text{ accepts } \langle M, \langle M \rangle \rangle. \end{cases}$$

Now let's see what happens if we run D on $\langle D \rangle$ (i.e., on its encoding!).

Two possible outcomes (since it's a decider):

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w, \\ \text{reject} & \text{otherwise (if } M \text{ rejects or loops on } w \text{).} \end{cases}$$

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } H \text{ rejects } \langle M, \langle M \rangle \rangle, \\ \text{reject} & \text{if } H \text{ accepts } \langle M, \langle M \rangle \rangle. \end{cases}$$

Now let's see what happens if we run D on $\langle D \rangle$ (i.e., on its encoding!).

Two possible outcomes (since it's a decider):

- D accepts $\langle D \rangle$:

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w, \\ \text{reject} & \text{otherwise (if } M \text{ rejects or loops on } w \text{).} \end{cases}$$

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } H \text{ rejects } \langle M, \langle M \rangle \rangle, \\ \text{reject} & \text{if } H \text{ accepts } \langle M, \langle M \rangle \rangle. \end{cases}$$

Now let's see what happens if we run D on $\langle D \rangle$ (i.e., on its encoding!).

Two possible outcomes (since it's a decider):

- D accepts $\langle D \rangle$:

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w, \\ \text{reject} & \text{otherwise (if } M \text{ rejects or loops on } w \text{).} \end{cases}$$

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } H \text{ rejects } \langle M, \langle M \rangle \rangle, \\ \text{reject} & \text{if } H \text{ accepts } \langle M, \langle M \rangle \rangle. \end{cases}$$

Now let's see what happens if we run D on $\langle D \rangle$ (i.e., on its encoding!).

Two possible outcomes (since it's a decider):

- D accepts $\langle D \rangle$:

This happens if H rejects $\langle D, \langle D \rangle \rangle$,

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w, \\ \text{reject} & \text{otherwise (if } M \text{ rejects or loops on } w \text{).} \end{cases}$$

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } H \text{ rejects } \langle M, \langle M \rangle \rangle, \\ \text{reject} & \text{if } H \text{ accepts } \langle M, \langle M \rangle \rangle. \end{cases}$$

Now let's see what happens if we run D on $\langle D \rangle$ (i.e., on its encoding!).

Two possible outcomes (since it's a decider):

- D **accepts** $\langle D \rangle$:

This happens if H rejects $\langle D, \langle D \rangle \rangle$, which from the definition of H happens when D **rejects** $\langle D \rangle$

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w, \\ \text{reject} & \text{otherwise (if } M \text{ rejects or loops on } w \text{).} \end{cases}$$

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } H \text{ rejects } \langle M, \langle M \rangle \rangle, \\ \text{reject} & \text{if } H \text{ accepts } \langle M, \langle M \rangle \rangle. \end{cases}$$

Now let's see what happens if we run D on $\langle D \rangle$ (i.e., on its encoding!).

Two possible outcomes (since it's a decider):

- D accepts $\langle D \rangle$:

This happens if H rejects $\langle D, \langle D \rangle \rangle$, which from the definition of H happens when D rejects $\langle D \rangle$

- D rejects $\langle D \rangle$:

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w, \\ \text{reject} & \text{otherwise (if } M \text{ rejects or loops on } w). \end{cases}$$

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } H \text{ rejects } \langle M, \langle M \rangle \rangle, \\ \text{reject} & \text{if } H \text{ accepts } \langle M, \langle M \rangle \rangle. \end{cases}$$

Now let's see what happens if we run D on $\langle D \rangle$ (i.e., on its encoding!).

Two possible outcomes (since it's a decider):

- D accepts $\langle D \rangle$:

This happens if H rejects $\langle D, \langle D \rangle \rangle$, which from the definition of H happens when D rejects $\langle D \rangle$

- D rejects $\langle D \rangle$:

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w, \\ \text{reject} & \text{otherwise (if } M \text{ rejects or loops on } w \text{).} \end{cases}$$

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } H \text{ rejects } \langle M, \langle M \rangle \rangle, \\ \text{reject} & \text{if } H \text{ accepts } \langle M, \langle M \rangle \rangle. \end{cases}$$

Now let's see what happens if we run D on $\langle D \rangle$ (i.e., on its encoding!).

Two possible outcomes (since it's a decider):

- D accepts $\langle D \rangle$:

This happens if H rejects $\langle D, \langle D \rangle \rangle$, which from the definition of H happens when D rejects $\langle D \rangle$

- D rejects $\langle D \rangle$:

This happens if H accepts $\langle D, \langle D \rangle \rangle$,

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w, \\ \text{reject} & \text{otherwise (if } M \text{ rejects or loops on } w). \end{cases}$$

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } H \text{ rejects } \langle M, \langle M \rangle \rangle, \\ \text{reject} & \text{if } H \text{ accepts } \langle M, \langle M \rangle \rangle. \end{cases}$$

Now let's see what happens if we run D on $\langle D \rangle$ (i.e., on its encoding!).

Two possible outcomes (since it's a decider):

- D accepts $\langle D \rangle$:

This happens if H rejects $\langle D, \langle D \rangle \rangle$, which from the definition of H happens when D rejects $\langle D \rangle$

- D rejects $\langle D \rangle$:

This happens if H accepts $\langle D, \langle D \rangle \rangle$, which from the definition of H happens when D accepts $\langle D \rangle$

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w, \\ \text{reject} & \text{otherwise (if } M \text{ rejects or loops on } w). \end{cases}$$

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } H \text{ rejects } \langle M, \langle M \rangle \rangle, \\ \text{reject} & \text{if } H \text{ accepts } \langle M, \langle M \rangle \rangle. \end{cases}$$

Now let's see what happens if we run D on $\langle D \rangle$ (i.e., on its encoding!).

Two possible outcomes (since it's a decider):

- D accepts $\langle D \rangle$:

This happens if H rejects $\langle D, \langle D \rangle \rangle$, which from the definition of H happens when D rejects $\langle D \rangle$

- D rejects $\langle D \rangle$:

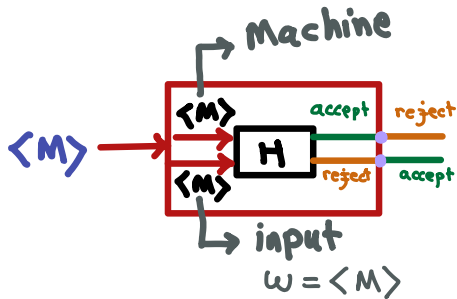
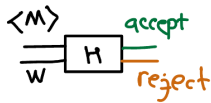
This happens if H accepts $\langle D, \langle D \rangle \rangle$, which from the definition of H happens when D accepts $\langle D \rangle$

Contradiction! Therefore H cannot exist and thus A_{TM} is undecidable!

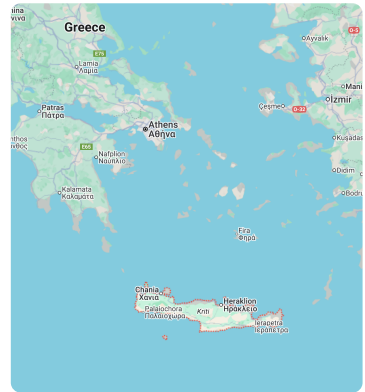
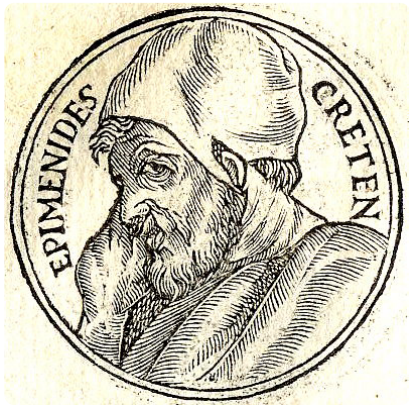
Visualizing the proof

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w, \\ \text{reject} & \text{otherwise (if } M \text{ rejects or loops on } w). \end{cases}$$

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } H \text{ rejects } \langle M, \langle M \rangle \rangle, \\ \text{reject} & \text{if } H \text{ accepts } \langle M, \langle M \rangle \rangle. \end{cases}$$



$D(\langle D \rangle) \Rightarrow \text{contradiction!}$
 input to D is $\langle D \rangle$!



600 BC

They fashioned a tomb for thee, O holy and high one
The Cretans, always liars, evil beasts, idle bellies!
But thou art not dead: thou livest and abidest forever,
For in thee we live and move and have our being.

— Epimenides, Cretica

The Halting Problem HALT_{TM}

$$\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ halts on input } w\}.$$

- Set of all (encoded) TM-input pairs such that TM **halt** on the input
- Equivalently (yes/no question):

given *any* TM M and *any* input string w , does M **halt** on w ?

The Halting Problem is Recognizable

$\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ halts on input } w\}.$

Theorem

HALT_{TM} is recognizable.

- **Proof:**
 - Given $\langle M, w \rangle$, we build another TM R that simulate M on w :

The Halting Problem is Recognizable

$\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ halts on input } w\}.$

Theorem

HALT_{TM} is recognizable.

- **Proof:**

- Given $\langle M, w \rangle$, we build another TM R that simulate M on w :
- If M halts (accept or reject), then R accepts.

The Halting Problem is Recognizable

$\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ halts on input } w\}.$

Theorem

HALT_{TM} is recognizable.

• Proof:

- Given $\langle M, w \rangle$, we build another TM R that simulate M on w :
- If M halts (accept or reject), then R accepts.
- (otherwise, the R also loops since it's simulating M)

The Halting Problem is Undecidable (Proof Sketch)

Suppose HALT_{TM} were decidable ...

```
def HALT(M_code, w):  
    # hypothetical decider  
    # returns True iff M halts on input w, otherwise returns False  
    ...  
  
def D(M_code):  
    if HALT(M_code, M_code):  
        while True:  
            pass      # loop forever  
    else:  
        accept()      # halt immediately
```

The Halting Problem is Undecidable (Proof Sketch)

Suppose HALT_{TM} were decidable ...

```
def HALT(M_code, w):  
    # hypothetical decider  
    # returns True iff M halts on input w, otherwise returns False  
    ...  
  
def D(M_code):  
    if HALT(M_code, M_code):  
        while True:  
            pass      # loop forever  
    else:  
        accept()      # halt immediately
```

Now consider running:

$D(D_code)$

The Halting Problem is Undecidable (Proof Sketch)

Suppose HALT_{TM} were decidable ...

```
def HALT(M_code, w):  
    # hypothetical decider  
    # returns True iff M halts on input w, otherwise returns False  
    ...  
  
def D(M_code):  
    if HALT(M_code, M_code):  
        while True:  
            pass      # loop forever  
    else:  
        accept()      # halt immediately
```

Now consider running:

$D(D_code)$

- If $\text{HALT}(D_code, D_code)$ is True $\rightarrow$ D loops forever
- If $\text{HALT}(D_code, D_code)$ is False $\rightarrow$ D halts

The Halting Problem is Undecidable (Proof Sketch)

Suppose HALT_{TM} were decidable ...

```
def HALT(M_code, w):  
    # hypothetical decider  
    # returns True iff M halts on input w, otherwise returns False  
    ...  
  
def D(M_code):  
    if HALT(M_code, M_code):  
        while True:  
            pass      # loop forever  
    else:  
        accept()      # halt immediately
```

Now consider running:

$D(D_code)$

- If $\text{HALT}(D_code, D_code)$ is True $\rightarrow$ D loops forever
- If $\text{HALT}(D_code, D_code)$ is False $\rightarrow$ D halts

Contradiction. Therefore HALT_{TM} is undecidable.

The Halting Problem is Undecidable

$$\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ halts on input } w\}.$$

Proof #1: (direct proof using a self-referential argument)

Suppose HALT_{TM} is decidable. Then there exists a *decider* TM H that:

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ halts on } w, \\ \text{reject} & \text{if } M \text{ loops on } w. \end{cases}$$

Now let's build **another** TM that takes as input $\langle M \rangle$, runs H on $\langle M, \langle M \rangle \rangle$, and does the following:

$$D(\langle M \rangle) = \begin{cases} \text{halt (accept or reject)} & \text{if } H \text{ rejects } \langle M, \langle M \rangle \rangle, \\ \text{loop} & \text{if } H \text{ accepts } \langle M, \langle M \rangle \rangle. \end{cases}$$

Now let's see what happens if we run D on $\langle D \rangle$ (i.e., on its encoding!):

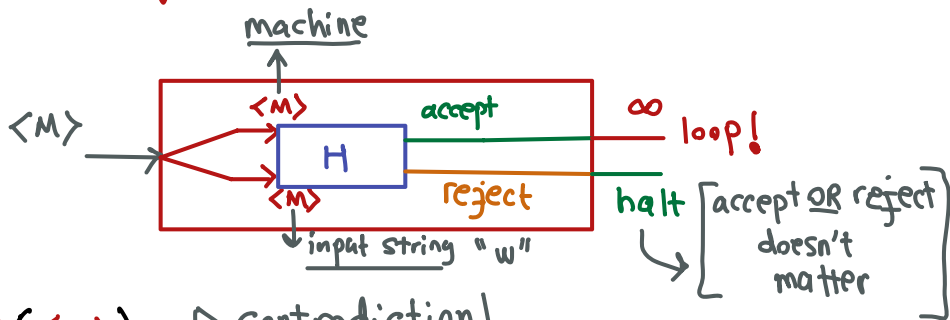
- If H says D halts on $\langle D \rangle$ then, by construction, D must loop!
- If H says D loops on $\langle D \rangle$ then, by construction, D must halt!

Contradiction! Therefore the decider H cannot exist!

Visualizing the proof

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ halts on } w, \\ \text{reject} & \text{otherwise if } M \text{ loops on } w. \end{cases}$$

$$D(\langle M \rangle) = \begin{cases} \text{halt} & \text{if } H \text{ rejects } \langle M, \langle M \rangle \rangle, \\ \text{loop} & \text{if } H \text{ accepts } \langle M, \langle M \rangle \rangle. \end{cases}$$



$D(\langle D \rangle) \Rightarrow \text{contradiction!}$

Second Proof of Undecidability of the Halting Problem using Reducibility

Second Proof of Undecidability of the Halting Problem

Idea: Use a known undecidable problem to show **ours** is undecidable!

Second Proof of Undecidability of the Halting Problem

Idea: Use a known undecidable problem to show **ours** is undecidable!

Proof #2: (using **reduction**)

Sketch:

Second Proof of Undecidability of the Halting Problem

Idea: Use a known undecidable problem to show **ours** is undecidable!

Proof #2: (using **reduction**)

Sketch:

- 1 Suppose **HALT_{TM}** is decidable (to show contradiction).

Second Proof of Undecidability of the Halting Problem

Idea: Use a known undecidable problem to show **ours** is undecidable!

Proof #2: (using **reduction**)

Sketch:

- 1 Suppose HALT_{TM} is decidable (to show contradiction).
- 2 Therefore HALT_{TM} has a decider (i.e., a TM that decides it and never loops).

Second Proof of Undecidability of the Halting Problem

Idea: Use a known undecidable problem to show **ours** is undecidable!

Proof #2: (using **reduction**)

Sketch:

- 1 Suppose **HALT_{TM}** is decidable (to show contradiction).
- 2 Therefore **HALT_{TM}** has a decider (i.e., a TM that decides it and never loops).
- 3 We show that using the **Halting decider** (assuming it exists), one could build a **decider** for **the Acceptance Problem A_{TM}** .

Second Proof of Undecidability of the Halting Problem

Idea: Use a known undecidable problem to show **ours** is undecidable!

Proof #2: (using **reduction**)

Sketch:

- 1 Suppose **HALT_{TM}** is decidable (to show contradiction).
- 2 Therefore **HALT_{TM}** has a decider (i.e., a TM that decides it and never loops).
- 3 We show that using the **Halting decider** (assuming it exists), one could build a **decider** for **the Acceptance Problem A_{TM}** .
- 4 But we have already proved A_{TM} is undecidable $\Rightarrow$ contradiction!

Second Proof of Undecidability of the Halting Problem

Idea: Use a known undecidable problem to show **ours** is undecidable!

Proof #2: (using **reduction**)

Sketch:

- 1 Suppose **HALT_{TM}** is decidable (to show contradiction).
- 2 Therefore **HALT_{TM}** has a decider (i.e., a TM that decides it and never loops).
- 3 We show that using the **Halting decider** (assuming it exists), one could build a **decider** for **the Acceptance Problem A_{TM}** .
- 4 But we have already proved A_{TM} is undecidable $\Rightarrow$ contradiction!
- 5 Therefore, **HALT_{TM}** cannot have a decider and is undecidable.

The Halting Problem is Undecidable (Proof #2 Sketch)

Suppose HALT_{TM} were decidable ...

```
def HALT(M_code, w):  
    # hypothetical decider  
    # returns True iff M halts on w  
    ...  
  
def DECIDE_ATM(M_code, w):  
    if not HALT(M_code, w):  
        reject()  
    else:  
        result = simulate(M_code, w)  
        if result == "accept":  
            accept()  
        else:  
            reject()
```

The Halting Problem is Undecidable (Proof #2 Sketch)

Suppose HALT_{TM} were decidable ...

```
def HALT(M_code, w):  
    # hypothetical decider  
    # returns True iff M halts on w  
    ...  
  
def DECIDE_ATM(M_code, w):  
    if not HALT(M_code, w):  
        reject()  
    else:  
        result = simulate(M_code, w)  
        if result == "accept":  
            accept()  
        else:  
            reject()
```

This would decide $\text{A}_{\text{TM}} \Rightarrow$ contradiction.

Proof #2: (using **reduction**)

Suppose **HALT**_{TM} is decidable. Let H be its decider TM. Therefore:

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ halts on } w, \\ \text{reject} & \text{if } M \text{ loops on } w. \end{cases}$$

We use H (as a *subroutine*) to build a **decider** D for A_{TM} .

⁵That is: M loops on w .

⁶That is: M halts on w . After this point, we know M **does NOT** loop on w .

⁷That is: accept if M accepts w , and reject if M rejects w .

⁸That is: it accepts/rejects correctly and never loops!

Proof #2: (using **reduction**)

Suppose **HALT_{TM}** is decidable. Let H be its decider TM. Therefore:

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ halts on } w, \\ \text{reject} & \text{if } M \text{ loops on } w. \end{cases}$$

We use H (as a *subroutine*) to build a **decider D for A_{TM}** .

On input $\langle M, w \rangle$:

⁵That is: M loops on w .

⁶That is: M halts on w . After this point, we know M **does NOT** loop on w .

⁷That is: accept if M accepts w , and reject if M rejects w .

⁸That is: it accepts/rejects correctly and never loops!

Proof #2: (using **reduction**)

Suppose **HALT_{TM}** is decidable. Let H be its decider TM. Therefore:

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ halts on } w, \\ \text{reject} & \text{if } M \text{ loops on } w. \end{cases}$$

We use H (as a *subroutine*) to build a **decider D for A_{TM}** .

On input $\langle M, w \rangle$:

- D first calls $H(\langle M, w \rangle)$

⁵That is: M loops on w .

⁶That is: M halts on w . After this point, we know M **does NOT** loop on w .

⁷That is: accept if M accepts w , and reject if M rejects w .

⁸That is: it accepts/rejects correctly and never loops!

Proof #2: (using **reduction**)

Suppose **HALT_{TM}** is decidable. Let H be its decider TM. Therefore:

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ halts on } w, \\ \text{reject} & \text{if } M \text{ loops on } w. \end{cases}$$

We use H (as a *subroutine*) to build a **decider D for A_{TM}** .

On input $\langle M, w \rangle$:

- D first calls $H(\langle M, w \rangle)$
- If $H(\langle M, w \rangle) = \text{rejects}^5 \Rightarrow D$ rejects.

⁵That is: M loops on w .

⁶That is: M halts on w . After this point, we know M **does NOT** loop on w .

⁷That is: accept if M accepts w , and reject if M rejects w .

⁸That is: it accepts/rejects correctly and never loops!

Proof #2: (using **reduction**)

Suppose **HALT_{TM}** is decidable. Let H be its decider TM. Therefore:

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ halts on } w, \\ \text{reject} & \text{if } M \text{ loops on } w. \end{cases}$$

We use H (as a *subroutine*) to build a **decider D for A_{TM}** .

On input $\langle M, w \rangle$:

- D first calls $H(\langle M, w \rangle)$
- If $H(\langle M, w \rangle) = \text{rejects}$ ⁵ $\Rightarrow D$ rejects.
- Else if $H(\langle M, w \rangle) = \text{accept}$ ⁶ $\Rightarrow D$ simulates M on input w , and accept/reject accordingly.⁷

⁵That is: M loops on w .

⁶That is: M halts on w . After this point, we know M **does NOT** loop on w .

⁷That is: accept if M accepts w , and reject if M rejects w .

⁸That is: it accepts/rejects correctly and never loops!

Proof #2: (using **reduction**)

Suppose **HALT_{TM}** is decidable. Let H be its decider TM. Therefore:

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ halts on } w, \\ \text{reject} & \text{if } M \text{ loops on } w. \end{cases}$$

We use H (as a *subroutine*) to build a **decider D for A_{TM}** .

On input $\langle M, w \rangle$:

- D first calls $H(\langle M, w \rangle)$
- If $H(\langle M, w \rangle) = \text{rejects}$ ⁵ $\Rightarrow D$ rejects.
- Else if $H(\langle M, w \rangle) = \text{accept}$ ⁶ $\Rightarrow D$ simulates M on input w , and accept/reject accordingly.⁷

⁵That is: M loops on w .

⁶That is: M halts on w . After this point, we know M **does NOT** loop on w .

⁷That is: accept if M accepts w , and reject if M rejects w .

⁸That is: it accepts/rejects correctly and never loops!

Proof #2: (using **reduction**)

Suppose **HALT_{TM}** is decidable. Let H be its decider TM. Therefore:

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ halts on } w, \\ \text{reject} & \text{if } M \text{ loops on } w. \end{cases}$$

We use H (as a *subroutine*) to build a **decider D for A_{TM}** .

On input $\langle M, w \rangle$:

- D first calls $H(\langle M, w \rangle)$
- If $H(\langle M, w \rangle) = \text{rejects}^5 \Rightarrow D$ rejects.
- Else if $H(\langle M, w \rangle) = \text{accept}^6 \Rightarrow D$ simulates M on input w , and accept/reject accordingly.⁷

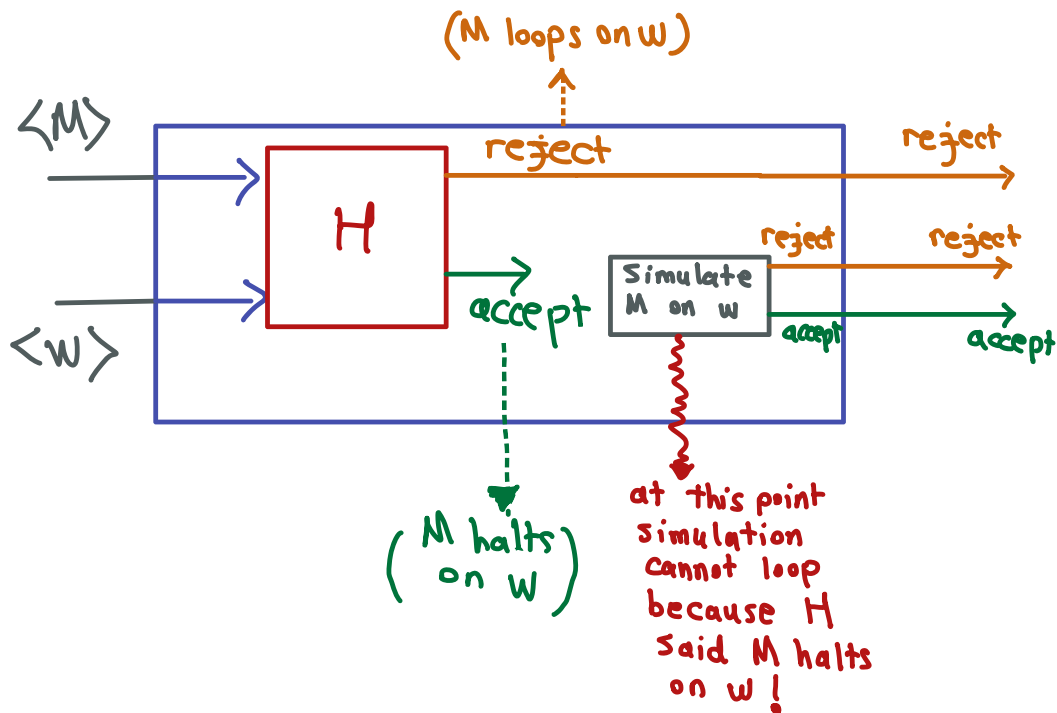
D is a decider⁸ for $A_{\text{TM}} \Rightarrow$ **Contradiction**, because A_{TM} is undecidable!
Therefore the decider H cannot exist and thus **HALT_{TM}** is undecidable.

⁵That is: M loops on w .

⁶That is: M halts on w . After this point, we know M **does NOT** loop on w .

⁷That is: accept if M accepts w , and reject if M rejects w .

⁸That is: it accepts/rejects correctly and never loops!



Reductions and Reducibility

- **Problem A** is **reducible** to **Problem B**:
An algorithm for solving **Problem B** can be used as a *subroutine* in an algorithm for solving **Problem A**.
- $\Rightarrow$ Informally, **Problem A** cannot be “harder” than **Problem B**.

Problem A “ $\leq$ ” Problem B

- Therefore, if **Problem A** is “very hard”, then so is **Problem B**.
- In the second proof, we showed that A_{TM} is reducible to HALT_{TM} .

Unrecognizable Problems!

Examples of Unrecognizable Languages

- Complement of language $L \subseteq \Sigma^*$ is the set of all strings from Σ^* that are not in L , i.e.,

$$\bar{L} = \{w \in \Sigma^* \mid w \notin L\}$$

Theorem (You'll prove this in A2!)

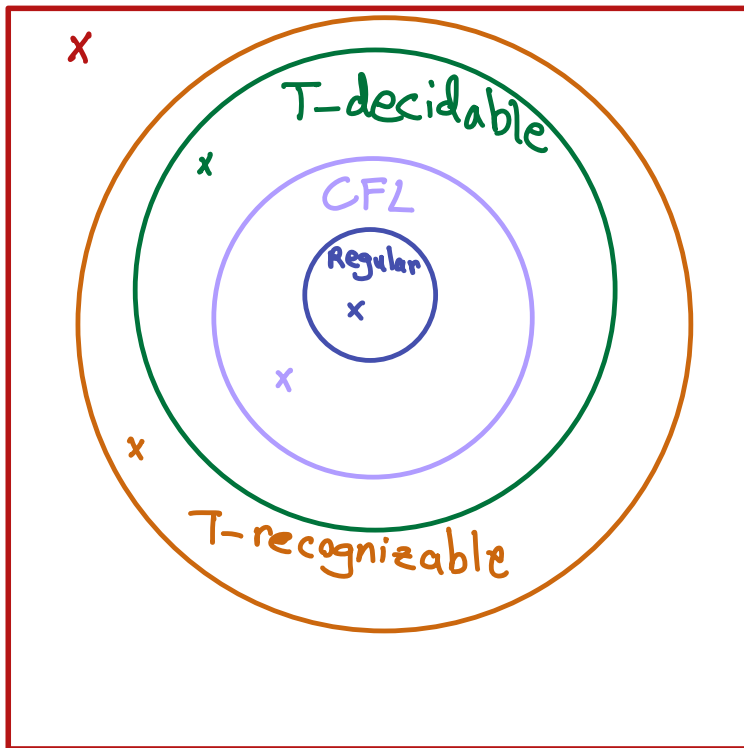
L is *decidable* if and only if **both L and its complement** are *recognizable*.

In A2 you will also prove:

$\overline{A_{TM}}$ and $\overline{HALT_{TM}}$ (i.e., complements of the Acceptance and Halting problems) are unrecognizable!

Big Picture – One Last Time!

x even # of 1's
x $\{a^n b^n \mid n \geq 0\}$
x $\{a^n b^n c^n \mid n \geq 0\}$
x HALT_{TM}
x $\overline{\text{HALT}_{\text{TM}}}$



Pale Blue Dot

Carl Sagan



From this distant vantage point, the Earth might not seem of any particular interest. But for us, it's different. Consider again that dot. That's here. That's home. That's us. On it everyone you love, everyone you know, everyone you ever heard of, every human being who ever was, lived out their lives. The aggregate of our joy and suffering, thousands of confident religions, ideologies, and economic doctrines, every hunter and forager, every hero and coward, every creator and destroyer of civilization, every king and peasant, every young couple in love, every mother and father, hopeful child, inventor and explorer, every teacher of morals, every corrupt politician, every "superstar", every "supreme leader", every saint and sinner in the history of our species lived there – on a mote of dust suspended in a sunbeam.